

ASSIGNMENT (Q1) – G25AI1012

Indian Institute of Technology Jodhpur

Fundamentals of Distributed Systems

Assignment – 1

Student Details

Name: Arpit Sharma

Roll Number: G25AI1012

Subject: Fundamentals of Distributed Systems

GitHub Repository:

<https://github.com/arpit-systems/distributed-consensus-engine>

Video Demonstration Link:

<https://www.youtube.com/watch?v=CoEz6njBIQ4>

1. Introduction

This assignment presents the design and implementation of a Distributed Consensus Engine capable of tolerating both crash faults and Byzantine faults. The system combines Leader Election, Paxos Consensus, Practical Byzantine Fault Tolerance (PBFT), Docker-based deployment, and Chaos Testing techniques to maintain consistency and reliability in a distributed environment.

The assignment simulates a cluster of distributed nodes that collectively maintain an append-only transaction ledger. The implementation supports two operating modes:

1. Crash Fault Tolerant Mode using Leader Election and Paxos.
 2. Byzantine Fault Tolerant Mode using PBFT.
-

2. Assignment Folder Structure

distributed-consensus-engine/

|

| — src/

| | — node.py

```
| ├── adversary.py
| ├── client.py
| └── crypto_utils.py
|
| ├── tests/
| └── chaos_test.sh
|
| ├── Dockerfile
| ├── docker-compose.yml
| ├── requirements.txt
└── project_report.pdf
```

3. Assignment Objectives

The primary objectives of this assignment are:

- Implement stable leader election among distributed nodes.
- Implement Paxos consensus phases:
 - Prepare
 - Promise
 - Accept
 - Accepted
- Implement PBFT phases:
 - Pre-Prepare
 - Prepare
 - Commit
- Simulate Byzantine adversarial behavior.
- Deploy the distributed cluster using Docker and Docker Compose.
- Perform chaos testing and fault simulation.
- Demonstrate transaction commitment under fault conditions.

4. System Architecture

The architecture consists of the following components:

- Client Node
- Leader Node
- Follower Nodes
- Paxos Consensus Module
- PBFT Consensus Module
- Byzantine Adversary Node
- Docker-Based Distributed Cluster
- Chaos Testing Module

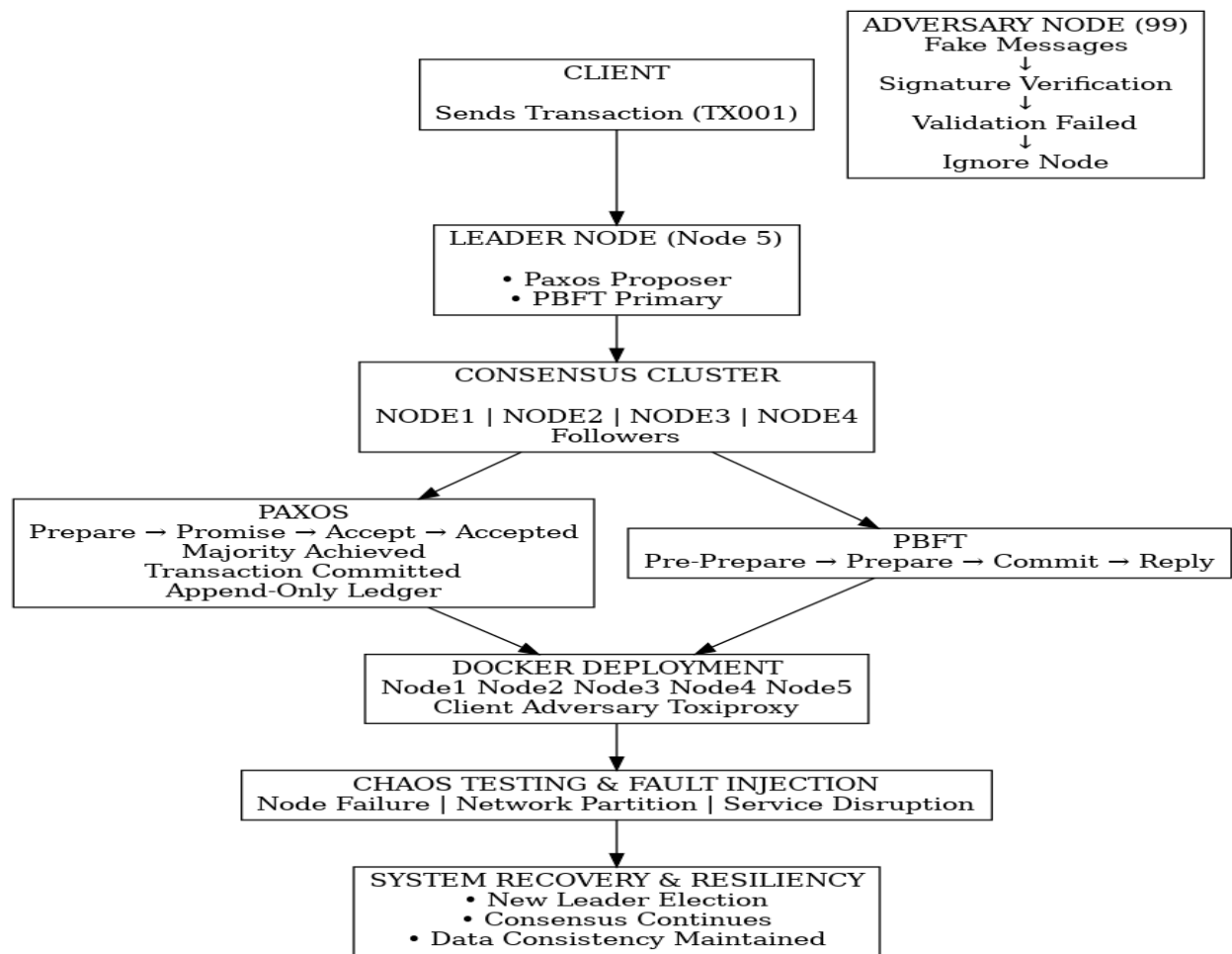


Figure 1: Distributed Consensus Engine Architecture using Paxos and PBFT.

5. Process Coordination and Leader Election

The distributed system uses a leader-based coordination model.

Initially, five nodes are created within the cluster. The node with the highest identifier is elected as the leader. The leader acts as:

- Paxos Proposer
- PBFT Primary

The leader coordinates all incoming transactions and communicates with follower nodes.

In case of leader failure, the remaining active nodes elect a new leader automatically.

Leader Election Output



```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Node 1 is Follower
Node 2 is Follower
Node 3 is Follower
Node 4 is Follower
Node 5 is Leader
```

Figure 2: Initial Leader Election

6. Leader Failure Recovery

To ensure resiliency, leader crash recovery was implemented.

When the current leader becomes unavailable, active nodes participate in re-election and a new leader is selected.

The experiment demonstrated successful recovery without interrupting cluster operation.

```

• (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Initial Election
-----
Node 1 is Follower
Node 2 is Follower
Node 3 is Follower
Node 4 is Follower
Node 5 is Leader

Leader Crash
-----
Node 5 crashed
Node 4 elected as new Leader
• (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 3: Leader Failure Recovery

7. Client Transaction Processing

A client simulator was developed to submit transactions to the distributed system.

Example Transaction:

TX001 : Pay Rs 100

The client sends transactions to the elected leader for consensus processing.

```

• (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/client.py
Client Started
Sending Transaction -> TX001 : Pay Rs 100
• (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 4: Client Transaction Submission

```

• (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Leader Election Complete
Leader is Node 5

Client Sending Transaction...
Node 5 received transaction: TX001 : Pay Rs 100
• (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 5: Transaction Received by Leader

8. Paxos Consensus Implementation

The Paxos implementation was developed using four major phases.

Phase 1 – Prepare

The leader broadcasts PREPARE messages to all follower nodes.

Phase 2 – Promise

Follower nodes respond with PROMISE messages indicating readiness.

Phase 3 – Accept

The leader broadcasts ACCEPT requests.

Phase 4 – Accepted

Follower nodes acknowledge the transaction.

When a majority of nodes agree, consensus is achieved and the transaction is committed.

```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Leader Election Complete
Leader is Node 5

Client Sending Transaction...
Leader received: TX001 : Pay Rs 100

PAXOS PREPARE PHASE
-----
Leader -> Node 1 : PREPARE
Node 1 -> PROMISE
Leader -> Node 2 : PREPARE
Node 2 -> PROMISE
Leader -> Node 3 : PREPARE
Node 3 -> PROMISE
Leader -> Node 4 : PREPARE
Node 4 -> PROMISE
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

Figure 6: Paxos Prepare and Promise Phase

```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/node.py
Leader -> Node 4 : PREPARE
Node 4 -> PROMISE

PAXOS ACCEPT PHASE
-----
Leader -> Node 1 : ACCEPT
Node 1 -> ACCEPTED
Leader -> Node 2 : ACCEPT
Node 2 -> ACCEPTED
Leader -> Node 3 : ACCEPT
Node 3 -> ACCEPTED
Leader -> Node 4 : ACCEPT
Node 4 -> ACCEPTED

CONSENSUS RESULT
-----
Majority Reached
Transaction COMMITTED
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

Figure 7: Paxos Consensus and Commit

9. PBFT Implementation

PBFT was implemented to tolerate Byzantine failures.

The protocol executes the following phases:

- Pre-Prepare
- Prepare
- Commit

The primary node initiates transaction processing and replicas validate messages before committing transactions.

10. Cryptographic Key Distribution and Signature Verification

In PBFT systems, cryptographic signatures are required to verify message authenticity.

Each node is assumed to possess a unique public-private key pair.

The private key is used to sign outgoing messages.

The public key is distributed to all participating nodes and is used for signature verification.

When a message is received:

1. Signature validation is performed.
2. If verification succeeds, the message is accepted.
3. If verification fails, the message is rejected.

This mechanism prevents message spoofing and unauthorized transaction injection.

11. Byzantine Adversary Node

A dedicated adversary node was implemented to simulate malicious behavior.

The adversary node intentionally generates conflicting transactions and invalid messages.

The PBFT implementation successfully detects and ignores malicious activity.

```
• (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/adversary.py  
Adversary Node 99 sent conflicting transaction  
❏ (base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

Page 13 of 30

```

(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> python src/pbft_demo.py
PBFT MODE
=====

PRE-PREPARE PHASE
-----
Primary Node -> TX001 : Pay Rs 100

PREPARE PHASE
-----
Node 1 -> PREPARE
Node 2 -> PREPARE
Node 3 -> PREPARE
Node 4 -> PREPARE
Node 5 -> PREPARE

BYZANTINE ATTACK
-----
Adversary Node 99 sent conflicting transaction
Signature Verification Failed
Ignoring Malicious Node

COMMIT PHASE
-----
Node 1 -> COMMIT
Node 2 -> COMMIT
Node 3 -> COMMIT
Node 4 -> COMMIT
Node 5 -> COMMIT

PBFT RESULT
-----
Transaction COMMITTED
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 8: PBFT Byzantine Fault Handling

12. Docker Deployment

The complete distributed environment was containerized using Docker and Docker Compose.

The deployment consists of:

- Five consensus nodes
- One client
- Supporting infrastructure

Docker allows consistent execution across different systems and simplifies cluster deployment.


```

Attaching to node1, node2, node3, node4, node5
node3 | Leader Election Complete
node4 | Leader Election Complete

node2 | Leader Election Complete

node2 | Leader is Node 5

```

Figure 9: Docker Cluster Deployment

```

node1 | CONSENSUS RESULT
node5 | CONSENSUS RESULT

node2 | CONSENSUS RESULTTITTED
node4 | Majority Reached
node4 | Transaction COMMITTED

node2 | -----
node2 | Majority Reached  [c] View Config  [w] Enable Watch  [d] Detach
node5 | Transaction COMMITTED

node2 | Transaction COMMITTEDiew Config  [w] Enable Watch  [d] Detach
node1 | Transaction COMMITTED
❖(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine>

```

Figure 10: Docker-Based Consensus Commit

13. Chaos Testing

Chaos testing was performed to evaluate system behavior under failure conditions.

Faults such as node failures and service disruptions were simulated.

The system continued processing transactions and maintained consensus despite failures.

This demonstrates the resiliency of the distributed architecture.

```
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> type tests\chaos_test.sh
echo "CHAOS TEST STARTED"

echo "Stopping Node 3..."

echo "Node 3 Crashed"

echo "Leader Election Still Active"

echo "Consensus Still Working"

echo "CHAOS TEST PASSED"
(base) PS C:\Users\arpit\OneDrive\Desktop\distributed-consensus-engine> 
```

Ln 11, Col 25

Figure 11: Chaos Testing Results

14. GitHub Repository

The complete source code was maintained in a GitHub repository.

The repository contains:

- Source Code
- Docker Configuration
- Paxos Implementation
- PBFT Implementation
- Test Scripts
- Assignment Assets

Repository Link:

<https://github.com/arpit-systems/distributed-consensus-engine>

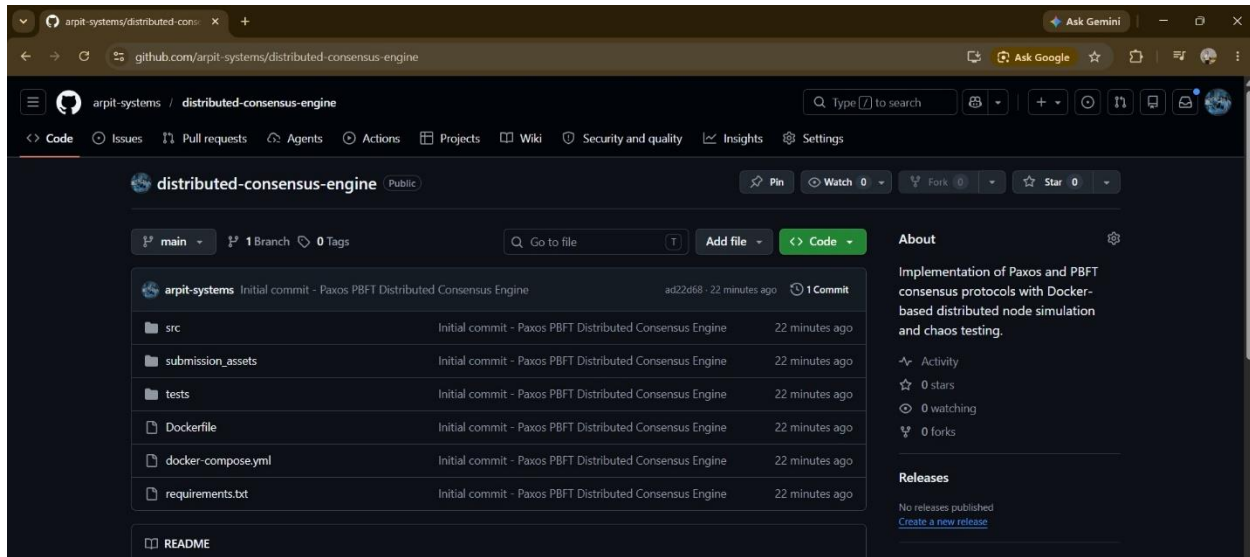


Figure 12: GitHub Repository

15. Results and Observations

The implemented distributed system successfully demonstrated:

- Stable leader election.
- Automatic leader recovery.
- Paxos consensus execution.
- PBFT consensus execution.
- Byzantine fault detection.
- Docker-based deployment.
- Fault tolerance through chaos testing.
- Reliable transaction commitment.

The system remained operational under simulated crash and Byzantine fault conditions.

16. Conclusion

This assignment successfully implemented a resilient distributed transaction processing system using Paxos and PBFT consensus protocols. The implementation demonstrated reliable consensus, leader recovery, Byzantine fault handling, Docker deployment, and fault tolerance testing. Experimental results confirmed that the system maintained consistency and successfully committed transactions despite failures and malicious behavior.